Chapter 3A — GPU and Accelerator Architecture Fundamentals

> *"The GPU is not a fast CPU. It is a fundamentally different machine — one that hides memory latency through massive parallelism rather than prediction, and achieves peak throughput only when you respect its constraints: aligned shapes, coalesced access, and fed Tensor Cores."*

Chapter Overview

This chapter establishes the hardware model that every optimization in Parts III–V relies on. You cannot reason about FlashAttention tiling, quantization bit-width selection, or tensor parallelism topology without first understanding the GPU execution engine, its memory hierarchy, and what the NVLink fabric actually does. By the end of this chapter you can read any GPU spec sheet analytically, predict which operations are compute-bound vs memory-bound on a given platform, and explain why SXM is mandatory for tensor parallelism.

Sections in this chapter:

- 3A.1 The GPU Memory Hierarchy — From Registers to HBM
- 3A.2 The Streaming Multiprocessor (SM) — Deep Dive
- 3A.3 Warp Scheduling, Occupancy, and Latency Hiding
- 3A.4 Tensor Cores — Generations, Formats, and Alignment Rules
- 3A.5 The NVLink and NVSwitch Interconnect
- 3A.6 PCIe vs SXM — Form Factor and What It Means for AI
- 3A.7 Alternative Accelerators — AMD, TPU, Gaudi
- 3A.8 How to Read a GPU Spec Sheet Analytically
- 3A.9 Key Takeaways and Review Questions

3A.1 The GPU Memory Hierarchy — From Registers to HBM

Every GPU optimization is ultimately about moving data through the memory hierarchy as efficiently as possible. The hierarchy has five levels, each with radically different bandwidth and capacity. The gap between levels determines which optimizations matter.

```
GPU MEMORY HIERARCHY — H100 SXM5: [SHIPPED]

  Level               | Capacity    | Bandwidth    | Latency     | Scope
  ---------------------------------------------------------------------------
  Registers           | ~256 KB/SM  | ~100 TB/s    | 0 cycles    | Per thread
  L1 / Shared Mem     | 228 KB/SM   | ~80 TB/s     | ~20 cycles  | Per SM block
  L2 Cache            | 50 MB       | ~12 TB/s     | ~150 cycles | All SMs
  HBM3 (GPU DRAM)     | 80 GB       | 3.35 TB/s    | ~400 cycles | All SMs
  PCIe 5.0 (CPU↔GPU)  | —           | ~128 GB/s    | ~1 μs       | Host access
  ---------------------------------------------------------------------------

  NVLink 4.0 (GPU↔GPU intra-node): 900 GB/s  [SHIPPED]
  InfiniBand NDR (inter-node):      ~50 GB/s   [SHIPPED]

BANDWIDTH RATIOS (what the numbers mean in practice):
  Registers vs L1:    100 TB/s / 80 TB/s = 1.25× — nearly the same tier
  L1 vs L2:           80 TB/s / 12 TB/s  = 6.7× — significant gap
  L2 vs HBM:          12 TB/s / 3.35 TB/s = 3.6× — meaningful gap
  HBM vs PCIe:        3.35 TB/s / 128 GB/s = 26× — huge gap
  HBM vs NVLink:      3.35 TB/s / 0.9 TB/s = 3.7× — NVLink is fast but not HBM
  HBM vs InfiniBand:  3.35 TB/s / 0.05 TB/s = 67× — IB is the slow lane

  The core performance engineering principle:
    Keep data as close to registers as possible.
    Every movement down the hierarchy costs bandwidth.
    FlashAttention exists to avoid moving attention matrices to HBM.
    Operator fusion exists to avoid HBM round-trips for elementwise ops.
    KV cache management (Chapter 11) is entirely about HBM capacity and BW.
```

3A.1.1 Shared Memory — The Programmer-Controlled Cache

```
SHARED MEMORY (SMEM) — THE KEY KERNEL OPTIMIZATION SURFACE:

  Physical: same SRAM as L1 cache on H100 (228 KB total per SM)
  Configurable split between L1 and SMEM (firmware/driver controlled)

  Default H100: 128 KB SMEM + 100 KB L1 (typical inference config)
  FlashAttention config: 228 KB SMEM + minimal L1 (maximize tile size)

  Why SMEM is critical for GEMM tiling:
    GEMM goal: compute C = A × B for large matrices.
    Naive approach: Load A[tile] and B[tile] from HBM for every output element.
```

```
      → Many HBM round-trips → memory-bound even for large GEMM

    Tiled approach: Load one A-tile and one B-tile into SMEM.
      → Compute all outputs that use these tiles → reuse amortizes HBM load cost.
      → SMEM bandwidth (~80 TB/s) feeds the math
      → HBM only accessed once per tile (not once per output element)

    Arithmetic Intensity with tiling:
      Without tiling: AI = 2 × M × N × K bytes / (M×K + K×N) ≈ 2K/N+K ≈ 2
FLOP/byte
      With SMEM tiling: AI = 2 × M × N × K / ((M×K + K×N)/tile_size)
                           = 2 × tile_size  FLOP/byte

    H100 tile_size target: 128-256 elements → AI = 256-512 FLOP/byte
    → Above H100 ridge point (295) → compute-bound → Tensor Cores fully fed

  The SMEM capacity constraint sets the maximum tile size:
    Tile bytes = tile_M × tile_K × bytes_per_element (A) + tile_K × tile_N × bytes
(B)
    At BF16: 228 KB / 2 bytes = 114K elements max in SMEM
    Square tile: sqrt(114K) ≈ 337 × 337 — in practice 128×128 or 256×128 tiles
```

3A.2 The Streaming Multiprocessor (SM) — Deep Dive

The SM is the fundamental compute unit of every NVIDIA GPU. Every performance optimization at the kernel level reduces to: how well am I using the SM's resources?

```
H100 SM INTERNAL ARCHITECTURE: [SHIPPED]

  132 SMs total (H100 SXM5)

  Per SM:
    4× Warp Schedulers
      Each warp scheduler issues 2 instructions per cycle
      Total: 8 instructions per cycle per SM

    128× FP32 CUDA cores (4 groups of 32)
      For elementwise operations, not GEMM

    4× 4th-generation Tensor Core units
      Each: WGMMA (warpgroup matrix multiply-accumulate) capable
      BF16 throughput: 256 BF16 FMAs/cycle per TC unit
      All 4 TCs: 1,024 BF16 FMAs/cycle/SM

    64× FP64 CUDA cores
    32× SFU (Special Function Units: sin, cos, exp, log, rsqrt)
    32× LD/ST units (Load/Store)

    228 KB SMEM (configurable split with L1)
    65,536× 32-bit registers (256 KB total)

  Instruction throughput per SM per cycle:
    FP32 CUDA cores (non-TC): 128 FP32 ops/cycle
    BF16 Tensor Core:         ~1,024 BF16 FMAs/cycle  (8× FP32!)
    INT8 Tensor Core:         ~2,048 INT8 ops/cycle   (16× FP32!)

  KEY POINT: Tensor Cores are why AI workloads run on GPUs.
  Without Tensor Cores: LLM training is not economically feasible.
  The ratio is not marginal — TCs deliver 8-16× the throughput of CUDA cores.
  Any kernel that fails to use Tensor Cores is leaving 87-94% of SM throughput
unused.
```

3A.2.1 Aggregate H100 Compute Calculation

```
VERIFYING H100 PEAK BF16 TFLOPS FROM FIRST PRINCIPLES:

  Per SM:     4 TC units × 256 BF16 FMAs/cycle = 1,024 FMAs/cycle
  All SMs:    132 SMs × 1,024 FMAs/cycle = 135,168 FMAs/cycle
  Clock:      1.98 GHz (boost clock)
```

```
  Per FLOP:   1 FMA = 2 FLOPs (1 multiply + 1 add)

  Peak BF16 = 135,168 × 1.98 × 10^9 × 2
            = 135,168 × 3.96 × 10^9
            = 535,265,280 × 10^9
            ≈ 535 TFLOPS  (per SM calculation)

  Published spec: 989 TFLOPS (with tensor core, dense)

  Gap explanation:
    NVIDIA's 989 TFLOPS figure uses maximum achievable throughput with
    optimal warpgroup scheduling, pipelining, and async WGMMA instruction.
    Our simplified calculation misses async instruction overlap.
    Practical sustained throughput: ~600-700 TFLOPS on real workloads (60-70% of
peak).
    The 989 TFLOPS is the arithmetic ceiling, not the sustained rate.
```

3A.3 Warp Scheduling, Occupancy, and Latency Hiding

The GPU's fundamental strategy for performance is radically different from a CPU. CPUs hide memory latency with deep out-of-order pipelines and branch prediction. GPUs hide latency with thread count — switching to another warp while one waits.

3A.3.1 The Warp Model — SIMT Execution

```
WARP = 32 THREADS EXECUTING IN LOCKSTEP (SIMT):

  All 32 threads in a warp execute the same instruction simultaneously.
  Each thread operates on different data (single instruction, multiple threads).

  When a warp issues a memory load to HBM:
    HBM latency: ~400 cycles
    The warp stalls — it cannot execute for 400 cycles.

  What happens during those 400 cycles?
    The SM's warp scheduler selects another READY warp and issues its instruction.
    → Latency is hidden by running another warp's work during the stall.
    → This requires having enough active warps to keep the schedulers busy.
    → This is OCCUPANCY.

WARP DIVERGENCE — The Cost of Branching:
  If threads in a warp take different branches:
```

```
    GPU serializes: execute true branch (false threads masked), then false branch.
    Cost: 2× execution time for 50/50 split. Proportional for other splits.

  Rule: Avoid if/else inside GPU kernels. Replace with arithmetic (branchless).
  LLM kernels are naturally branch-free (GEMM, attention, elementwise ops).
```

3A.3.2 Occupancy — The Latency Hiding Dial

```
OCCUPANCY = Active Warps per SM / Max Warps per SM (64 for H100)

Three resources limit occupancy — whichever is exhausted first sets the ceiling:

RESOURCE 1 — REGISTER PRESSURE:
  65,536 registers / SM (fixed)
  A kernel using 64 registers/thread:
    64 warps × 32 threads × 64 regs = 131,072 — exceeds capacity!
    Maximum warps at 64 regs/thread = 65,536 / (64 × 32) = 32 → 50% occupancy

  FlashAttention-2 uses ~200 registers/thread:
    Max warps = 65,536 / (200 × 32) = ~10 → ~16% occupancy
    BUT: FlashAttention is bandwidth-bound on attention, not register-bound.
    Low occupancy is acceptable because HBM traffic is the actual bottleneck.
    This is why reporting occupancy without arithmetic intensity is misleading.

RESOURCE 2 — SHARED MEMORY:
  228 KB SMEM / SM
  A kernel using 80 KB SMEM per block:
    228 / 80 = 2 concurrent blocks → low occupancy
  FlashAttention uses most of SMEM for KV tiles — low occupancy is the design.

RESOURCE 3 — THREAD BLOCK COUNT:
  Max 32 concurrent thread blocks per SM (H100 Hopper)
  Small blocks (32 threads = 1 warp): up to 32 concurrent → high occupancy
  Large blocks (1024 threads = 32 warps): 2 concurrent → lower occupancy

THE OCCUPANCY RULE OF THUMB:
  Memory-bound kernels: high occupancy critical — need many warps to hide 400-
cycle latency
    Target: ≥50% (≥32 active warps per SM)
  Compute-bound kernels: moderate occupancy sufficient — Tensor Cores need tiles,
not warps
    Target: ≥25% — latency hiding matters less when compute is the bottleneck
```

```
WRONG MENTAL MODEL: "Higher occupancy = better performance" — FALSE
CORRECT: "Sufficient occupancy to hide the dominant latency = good performance"

A FlashAttention kernel at 16% occupancy beats a naive attention kernel at 90%
occupancy because FlashAttention's HBM traffic is 5-10× lower per token.
```

3A.4 Tensor Cores — Generations, Formats, and Alignment Rules

Tensor Cores are the single most important hardware innovation in GPU AI compute. Understanding their alignment requirements is essential for kernel optimization.

3A.4.1 Tensor Core Generations

```
TENSOR CORE EVOLUTION:

  Gen | GPU      | Formats              | Key Innovation
  ──────────────────────────────────────────────────────────────────────
  1st | Volta    | FP16 only            | First TC — 8× FP16 vs CUDA cores
      | (V100)   | 16×16×16 tile        |
  ──────────────────────────────────────────────────────────────────────
  2nd | Ampere   | BF16, FP16, TF32,    | TF32: FP32 range + FP16 precision
      | (A100)   | INT8, INT4           | 2:4 structured sparsity (2× compute)
      |          | 16×16×16 tile        | BF16 native (important for training)
  ──────────────────────────────────────────────────────────────────────
  3rd | Hopper   | FP8 (E4M3+E5M2),     | FP8 native — 2× throughput vs BF16
      | (H100)   | BF16, TF32, INT8     | WGMMA: async warpgroup MMA instruction
      |          | 16×16×16 + async     | Thread Block Clusters across SMs
      |          |                      | Transformer Engine integration
  ──────────────────────────────────────────────────────────────────────
  4th | Blackwell| FP4 (NVFP4), FP6,    | FP4: 2× throughput vs FP8
      | (B200)   | FP8, BF16, TF32      | 2nd-gen Transformer Engine
      |          | 32×16×16 tile        | Block-level scaling for FP4/FP6/FP8
      |          |                      | 2.3× vs H100 FP8 peak compute
  ──────────────────────────────────────────────────────────────────────


CUMULATIVE PRECISION GAIN:
  A100 BF16: 312 TFLOPS  (baseline)
  H100 BF16: 989 TFLOPS  (3.2× vs A100 — TCs + process + wgmma)
  H100 FP8:  1979 TFLOPS (2× H100 BF16 — halved bit width)
  B200 FP8:  4500 TFLOPS (2.3× H100 FP8)
  B200 FP4:  9000 TFLOPS (2× B200 FP8)
```

```
  From A100 BF16 to B200 FP4: ~29× in four years.
```

3A.4.2 The MMA Instruction — How Tensor Cores Execute

```
MATRIX MULTIPLY-ACCUMULATE (MMA) OPERATION:
  D = A × B + C

  H100 WGMMA instruction (64×256×16 tile, BF16):
    A: [64×16] BF16 (in registers, one warpgroup = 128 threads)
    B: [16×256] BF16 (in SMEM — critical: B must be in SMEM for peak throughput)
    C: [64×256] FP32 (accumulator, in registers)
    Output D: [64×256] FP32

    FLOPs per WGMMA: 2 × 64 × 256 × 16 = 524,288 FLOPs
    Issued once per ~1 cycle (with pipelining)

  The async property:
    SM issues WGMMA instruction → Tensor Cores begin executing.
    SM simultaneously issues the NEXT data prefetch from SMEM.
    → Compute and data movement overlapped → hides tile load latency.
    This is why H100 sustains higher % of peak vs A100 despite similar SRAM BW.
```

3A.4.3 Tensor Core Alignment Requirements — The Silent Performance Killer

```
ALIGNMENT RULES — VIOLATING THESE SILENTLY KILLS PERFORMANCE:

H100 BF16 Tensor Core requirements:
  Matrix dimensions must be divisible by 16 for BF16/FP16
  Matrix dimensions must be divisible by 128 for FP8 (wgmma tile requirement)
  Memory access must be 128-byte aligned (coalesced loads)

What happens with misaligned shapes:
  ALIGNED:   [4096 × 4096] × [4096 × 4096] BF16
    → Full Tensor Core utilization, ~989 TFLOPS peak

  MISALIGNED: [4097 × 4097] × [4097 × 4097] BF16
    → CUDA compiler pads to next aligned boundary
    → Padding creates partial tiles
    → Partial tiles execute on CUDA cores, not Tensor Cores
    → Effective throughput: 30-40% of peak (a 2.5-3.3× slowdown)

  MISALIGNED EXAMPLE IN LLM SERVING (REAL ISSUE):
```

```
    Model with vocabulary size = 32,001 (prime — not divisible by 16)
    Final logit projection: [seq × d_model] × [d_model × 32001]
    Last column: 1 element instead of 16 → partial tile → CUDA core fallback
    Fix: pad vocabulary to 32,016 (next multiple of 16) — zero quality impact

DIAGNOSTIC: How to detect alignment problems:
  Nsight Compute → "Warp State Statistics" → sm_util vs tc_util
  If sm_util high but tc_util low: GEMM shapes are misaligned

  Quick check: print all GEMM dimensions in your model:
    for m, n, k in gemm_shapes:
        if m % 16 != 0 or n % 16 != 0 or k % 16 != 0:
            print(f"WARNING: Misaligned GEMM [{m}x{n}x{k}]")

TF32 — The A100 Training Optimization:
  Same 8-bit exponent as FP32 (preserves dynamic range)
  Only 10-bit mantissa (same as FP16 — reduced precision)
  Drops into FP32 training code transparently
  312 TFLOPS TF32 vs 19.5 TFLOPS FP64 = 16× speedup with ~1% quality loss
  PyTorch: torch.backends.cuda.matmul.allow_tf32 = True  (on by default in PyTorch
1.7+)
```

3A.4.4 The Transformer Engine — H100's Most Important Innovation for LLMs

```
TRANSFORMER ENGINE (H100+):

Problem: BF16 is the safe precision for LLM training but leaves FP8's 2×
         throughput on the table. Pure FP8 training causes gradient instability
         because different transformer layers have wildly different activation
ranges.

Solution: Per-layer automatic precision switching with hardware monitoring.

Mechanism (H100 Transformer Engine):
  Step 1: Monitor activation magnitude ranges per tensor during forward pass.
  Step 2: For each GEMM, determine if current range fits in FP8 without overflow.
  Step 3: If safe: issue FP8 GEMM → 1979 TFLOPS throughput.
          If unsafe: fall back to BF16 GEMM → 989 TFLOPS.
  Step 4: Accumulate in FP32 regardless for numerical stability.
  Step 5: Scale factor stored per tensor for gradient computation.

Result:
  ~1.9× average GEMM throughput vs pure BF16 (not full 2× due to some BF16
```

```
fallback)
  Quality: <0.2% degradation on standard LLM benchmarks
  Code change required: ZERO — framework/driver handles automatically

  Enable in PyTorch: transformer_engine.pytorch.fp8_autocast()
  Enable in Megatron-LM: --fp8-e4m3 flag

FP8 FORMAT PAIR — WHY TWO FORMATS:
  E4M3 (4-bit exponent, 3-bit mantissa):
    Range: ±448, higher precision
    Used for: weights and activations (stable during forward pass)

  E5M2 (5-bit exponent, 2-bit mantissa):
    Range: ±57344, lower precision
    Used for: gradients (wider range needed during backprop due to gradient
spikes)

  Together: enable full FP8 training without manual per-layer configuration.
```

3A.5 The NVLink and NVSwitch Interconnect

NVLink is the intra-node GPU interconnect that makes tensor parallelism viable. Understanding its architecture and bandwidth is essential for parallelism design.

3A.5.1 NVLink Architecture

```
NVLINK EVOLUTION:

  Version  | GPU     | Per-GPU BW  | Per-Link BW  | Links/GPU
  ──────────────────────────────────────────────────────────────
  3.0      | A100    | 600 GB/s    | 50 GB/s      | 12 links
  4.0      | H100    | 900 GB/s    | 50 GB/s      | 18 links  [SHIPPED]
  5.0      | B200    | 1,800 GB/s  | 100 GB/s     | 18 links  [SHIPPED]

H100 DGX NODE — 8 SXM GPUs + 4 NVSwitch chips:

  Per GPU: 18 NVLink 4.0 lanes × 50 GB/s = 900 GB/s bidirectional

  NVSwitch 3.0: 4 ASIC chips in the DGX chassis
    Each NVSwitch: non-blocking crossbar
    Each GPU connects to all 4 NVSwitches
    Any GPU → Any GPU: 900 GB/s full bisection bandwidth (no congestion)
```

```
  Total intra-node aggregate:
    8 GPUs × 900 GB/s = 7.2 TB/s aggregate NVLink bandwidth

WHY NVSWITCH INSTEAD OF POINT-TO-POINT NVLINK:
  With 8 GPUs and direct connections, you need C(8,2) = 28 GPU-pairs.
  Each pair needs 2 links (bidir) = 56 NVLink connections.
  8 GPUs × 18 lanes = 144 total lanes — not enough for full mesh.
  NVSwitch solves this: all 144 lanes go to 4 NVSwitch chips,
  which provide non-blocking all-to-all routing.

  NVSwitch also enables:
    SHARP (Scalable Hierarchical Aggregation and Reduction Protocol):
      In-switch AllReduce — some reduction happens inside the switch
      instead of on the GPU → reduces GPU compute overhead
    Multicast: one GPU sends to multiple GPUs simultaneously
```

3A.5.2 The TP Golden Rule — Why NVLink Topology Determines Parallelism Strategy

```
THE TENSOR PARALLELISM GOLDEN RULE:
  TP must run within the NVLink domain. Never extend TP across InfiniBand.

WHY THIS IS NON-NEGOTIABLE (LLaMA-3 70B, 80 layers, TP=8):

  Per-layer TP AllReduce: ~134 MB (activation shard per layer)
  AllReduces per step: 80 layers × 2 AllReduces = 160 per step

  Over NVLink (900 GB/s latency <0.1 μs):
    134 MB / 900 GB/s = 0.15 ms per AllReduce
    160 AllReduces = 24 ms per step  ← acceptable (step time ~200-400 ms)

  Over InfiniBand NDR (50 GB/s, ~1 μs latency):
    134 MB / 50 GB/s = 2.7 ms per AllReduce
    160 AllReduces = 432 ms per step  ← exceeds typical step time!

  TP over InfiniBand: 432 ms communication > 200 ms compute = 3× slower than
needed.

PRACTICAL RULES:
  TP group size = GPUs within ONE DGX/HGX node (8 GPUs via NVSwitch)
  PP = across nodes (InfiniBand, but only 2-4 messages per node boundary per step)
  DP = across node groups (InfiniBand, overlapped with backward pass)
```

```
  Never: TP=16 across 2 nodes. Always: PP=2 with TP=8 per node.

ALLREDUCE LATENCY COMPARISON:
  NVLink AllReduce (8 GPUs, 134 MB):
    Ring AllReduce time = 2 × (N-1)/N × msg_size / BW
                        = 2 × 7/8 × 134 MB / 900 GB/s
                        = 0.26 ms  (barely perceptible)

  InfiniBand AllReduce (same):
    = 2 × 7/8 × 134 MB / 50 GB/s
    = 4.7 ms  (catastrophic at 160 calls per step)
```

3A.5.3 GB200 NVL72 — The Rack as a Single Computer

```
GB200 NVL72 ARCHITECTURE: [SHIPPED]

  Configuration:
    36× Grace CPUs (ARM Neoverse V2, 72 cores each)
    72× B200 GPUs
    NVLink 5.0 connecting all 72 B200s
    NVLink-C2C: CPU↔GPU at 900 GB/s (7× PCIe Gen5)

  NVLink 5.0 per GPU: 1.8 TB/s (2× H100 NVLink 4.0)
  NVLink Switch ASICs: new generation connecting all 72 GPUs

  Aggregate rack bandwidth:
    72 × 1.8 TB/s = 129.6 TB/s aggregate NVLink bandwidth

  Unified memory:
    72 × 192 GB HBM3e = 13.8 TB total
    + 36 × 480 GB LPDDR5X (Grace CPU) = 17.3 TB CPU memory
    All coherently addressable → 30 TB unified memory pool

  Aggregate compute:
    72 × 4,500 TFLOPS (FP8) = 324 PFLOPS
    72 × 9,000 TFLOPS (FP4) = 648 PFLOPS

  THE ARCHITECTURAL IMPLICATION:
    All 72 GPUs are within ONE NVLink bandwidth domain.
    LLaMA-3.1 405B weights = 810 GB — fits entirely within NVL72 NVLink domain.
    TP=72 is now viable without InfiniBand for any weight matrix operation.
    No InfiniBand needed for AllReduce on weight GEMMs.
```

```
This is the "rack as a single computer" concept: the entire rack behaves
as one coherent 30 TB memory device with 130 TB/s bandwidth — more than
most previous entire cluster's combined interconnect bandwidth.
```

3A.6 PCIe vs SXM — Form Factor and What It Means for AI

```
PCIe vs SXM COMPARISON: [SHIPPED]

  Dimension          | PCIe H100                | SXM5 H100
  ------------------------------------------------------------------------
  Physical form       | Standard PCIe card       | Proprietary SXM baseboard
                      | Fits in any server       | Requires DGX/HGX chassis
  ------------------------------------------------------------------------
  GPU-to-GPU BW       | PCIe 5.0: ~128 GB/s      | NVLink 4.0: 900 GB/s
  (same server)       | (shared CPU bus)         | (via NVSwitch, dedicated)
  ------------------------------------------------------------------------
  GPU-to-CPU BW       | PCIe 5.0: ~128 GB/s      | NVLink-C2C: 900 GB/s
                      |                          | (GB200 superchip only)
  ------------------------------------------------------------------------
  HBM bandwidth       | ~3.35 TB/s               | 3.35 TB/s (full spec)
  (H100)              | (slight reduction due    |
                      | to power limits)         |
  ------------------------------------------------------------------------
  TDP                 | ~350W                    | ~700W
  ------------------------------------------------------------------------
  Sustained TFLOPS    | ~700-800 TFLOPS BF16     | ~989 TFLOPS BF16
  (thermal headroom)  | (lower at 350W)          |
  ------------------------------------------------------------------------
  Use case            | Cloud rental, inference  | Production training,
                      | without TP, cost-opt.    | TP-heavy serving
  ------------------------------------------------------------------------
  Price premium       | Baseline (~$25K/H100)    | ~2-3× ($60-80K/H100)

SXM IS MANDATORY WHEN:
  1. Tensor Parallelism required (model > GPU memory, or TP for throughput)
  2. Pipeline Parallelism with aggressive activation buffer management
  3. Training jobs where MFU target > 40% (requires NVLink AllReduce overlap)
  4. Inference with multi-GPU serving at production latency targets

PCIe IS VIABLE WHEN:
  1. Model fits on single GPU (e.g., 7B INT4: ~4 GB)
```

```
  2. Inference workloads without TP (batch serving at lower throughput)
  3. Cost-optimized cloud GPU rental for development
  4. Distillation or fine-tuning on small-batch workloads

AMD EQUIVALENT — OAM (Open Accelerator Module):
  MI300X uses OAM form factor (industry standard, not proprietary like SXM)
  Same concept: high-power board with dedicated power delivery
  Infinity Fabric between OAM modules: ~896 GB/s (similar to NVLink 4.0)
  Advantage: OAM is an open standard — multiple server vendors can build MI300X
nodes
```

3A.7 Alternative Accelerators — AMD, TPU, Gaudi

3A.7.1 AMD GPU — ROCm and CDNA Architecture

```
AMD AI GPU OVERVIEW — CDNA 3 (MI300X): [SHIPPED]

KEY DIFFERENCES FROM NVIDIA:
  Architecture: CDNA (Compute DNA) — pure compute, no graphics
  Programming model: ROCm HIP (C++, similar syntax to CUDA, different binary)
  Interconnect: Infinity Fabric (AMD's NVLink equivalent)
  Form factor: OAM (Open Accelerator Module)

MI300X KEY SPECS:
  Die architecture: 6 XCDs (eXtended Compute Dies) + 4 MCDs (Memory Cache Dies)
                    = 10 dies total on one package
                    (NOT "3 chiplets" — a common misconception in the industry)
  Total CUs: 304 Compute Units
  Peak BF16: ~1,307 TFLOPS
  HBM3: 192 GB (the competitive moat — 2.4× H100's 80 GB)
  HBM bandwidth: 5.3 TB/s (58% more than H100)
  TDP: 750W

WHY 192 GB IS THE KEY MI300X ADVANTAGE:
  LLaMA-3 70B in BF16 = 140 GB weights.
    H100 (80 GB): CANNOT FIT → requires TP=2 (2 GPUs, adds AllReduce overhead)
    MI300X (192 GB): FITS on 1 GPU → no TP needed, eliminates AllReduce per layer

  This eliminates:
    160 AllReduce calls per step (80 layers × 2)
    ~24 ms communication overhead per step (per our Chapter 14 analysis)
```

```
    Second GPU cost (~$25-60K depending on hardware)
    Serving complexity (1 process, not 2)

INFERENCE THROUGHPUT COMPARISON (70B FP16, batch=1):
  MI300X (single GPU): ~25 tokens/sec   (5.3 TB/s / 140 GB = 37.9 tok/s × ~65%
util)
  H100 × 2 (TP=2):     ~22 tokens/sec   (2 × 3.35 TB/s eff. minus AllReduce)
  MI300X wins on simplicity; H100 × 2 slightly faster at sustained large batch.

ROCm vs CUDA — THE SOFTWARE GAP:
  CUDA (NVIDIA): 15+ years of optimization, cuBLAS, cuDNN, CUTLASS, TRT-LLM
  ROCm (AMD): ~8 years, HIP (CUDA-compatible API), rocBLAS, MIOpen

  Gap is real but closing:
    PyTorch ROCm: production quality (both run identical PyTorch code)
    vLLM ROCm: production ready, 85-95% of CUDA performance
    TRT-LLM equivalent: MFMA (MI Fusion MoE Attention) — improving rapidly
    Custom kernels: still a gap for specialized attention variants
    Debugging tools: ROCm omniperf vs Nsight Compute — improving

  For standard LLM workloads (vLLM, PyTorch): AMD is viable.
  For custom kernel development: NVIDIA still has 30-40% workflow advantage.
```

3A.7.2 Google TPU — Architecture for Comparison

```
GOOGLE TPU v4 / v5p — KEY ARCHITECTURE POINTS:

  Design philosophy: systolic array optimized for GEMM and matrix ops.
    Unlike GPU (general parallel): TPU is deeply specialized for matrix multiply.
    Matrix Unit: 128×128 systolic array for each MXU

  v5p (2024): [SHIPPED for GCP customers]
    4,096 TPU chips per pod
    9 × 4,096 = 36,864 chips in a "supercomputer"
    459 PFLOPS per TPU pod (BFLOAT16)
    ICI interconnect: 4,800 GB/s per chip (6× H100 NVLink equivalent)
    Memory: 95 GB HBM2e per chip at 2.76 TB/s

ADVANTAGE vs GPU:
  ICI (Inter-Chip Interconnect): 4,800 GB/s vs H100's 900 GB/s NVLink
    → 5.3× faster intra-cluster interconnect
    → Allows larger tensor parallelism degree without interconnect bottleneck
```

```
  Cost: generally lower for Google Cloud training (Google's vertically integrated
cost)

DISADVANTAGE vs GPU:
  HBM bandwidth lower (2.76 TB/s vs H100 3.35 TB/s) — hurts LLM inference
  Only available on Google Cloud — no on-premise deployment
  XLA compiler required: no CUDA, different programming model
  Custom kernel support: much harder than CUDA (limited Pallas support)
  TTFT latency: ICI topology optimized for throughput, not latency
```

3A.7.3 Intel Gaudi — The Open Alternative

```
INTEL GAUDI 3 (2024): [SHIPPED]

  Architecture: Gaudi ASIC — dedicated AI accelerator
  Peak BF16: ~1,835 TFLOPS
  Memory: 128 GB HBM2e at 3.7 TB/s
  Network: 24× 200GbE RoCE v2 NICs embedded on chip
  TDP: 900W

KEY DIFFERENTIATOR:
  NICs embedded on-chip: no separate InfiniBand adapter needed.
  24 × 200GbE = 4.8 Tbps = ~600 GB/s usable bandwidth.
  For collective communication: uses standard RoCE over Ethernet (open ecosystem).

COMPETITIVE POSITION:
  Gaudi 3 approaches H100-class on standard benchmarks.
  Where Gaudi falls short: custom kernel ecosystem (no CUDA equivalent).
  Where Gaudi leads: open networking (no InfiniBand license cost).
  Primary market: cost-conscious enterprise buyers avoiding NVIDIA premiums.
```

3A.8 How to Read a GPU Spec Sheet Analytically

This section provides a structured method for evaluating any GPU spec sheet and converting marketing numbers into performance predictions for your workload.

```
GPU SPEC SHEET ANALYSIS FRAMEWORK:

STEP 1 — Extract the three critical numbers:
  Peak FLOPS (BF16 or FP8)  — the compute ceiling
  HBM bandwidth (TB/s)       — the memory ceiling
  HBM capacity (GB)          — the memory capacity constraint

  Calculate ridge point immediately:
    Ridge = Peak FLOPS / HBM bandwidth  [FLOP/byte]

STEP 2 — Classify your workload:
  Training large batch (GEMM-heavy):
    → Compute-bound workload, above ridge point
    → Higher FP8 TFLOPS matters more than higher HBM BW
    → Evaluate: FP8 TFLOPS and Tensor Core alignment support

  Inference decode (batch=1 to ~128):
    → Memory-BW-bound workload, well below ridge point
    → HBM bandwidth matters more than FP8 TFLOPS
    → Evaluate: HBM bandwidth, then HBM capacity for KV cache

  Long-context prefill (S > 10K tokens):
    → Near or above ridge point
    → Both compute and bandwidth matter
    → Evaluate: FP8 TFLOPS AND HBM bandwidth

STEP 3 — Check interconnect for parallelism requirements:
  Model size vs. HBM capacity determines TP requirement:
    Model BF16 size = N_params × 2 bytes
    If model_size > HBM_capacity: TP required
    TP degree = ceil(model_size / HBM_capacity)

  TP degree vs interconnect quality:
    TP within NVLink domain: viable
    TP across InfiniBand: catastrophic for latency
    If TP > 8: requires multi-node TP (expensive) or larger HBM

STEP 4 — Calculate decode throughput estimate:
  Theoretical max decode tokens/sec (batch=1, ignoring KV cache):
    = HBM_bandwidth / model_bytes
    = HBM_bandwidth / (N_params × 2 bytes)
  Practical (multiply by ~0.65-0.75 for real utilization):
    = theoretical × 0.70
```

```
  EXAMPLE CALCULATIONS:
    GPU          HBM BW    70B model decode    Practical (70%)
    A100 SXM4    2.0 TB/s  14.3 tok/s × 2GPU  ~20 tok/s (TP=2)
    H100 SXM5    3.35 TB/s 23.9 tok/s × 2GPU  ~33 tok/s (TP=2)
    H200 SXM5    4.8 TB/s  34.3 tok/s × 1GPU  ~24 tok/s (1 GPU — fits!)
    MI300X       5.3 TB/s  37.9 tok/s × 1GPU  ~27 tok/s (1 GPU — fits!)
    B200 SXM     8.0 TB/s  57.1 tok/s × 1GPU  ~40 tok/s (1 GPU — fits!)

STEP 5 — Evaluate precision support for your target quality:
  FP8 support: 2× throughput for compute-bound workloads
  Check: does the hardware support automatic calibration (Transformer Engine)?
  Check: does your software stack support FP8 on this hardware?
    NVIDIA H100: yes (TRT-LLM, vLLM, Megatron-LM all support TE)
    AMD MI300X: FP8 support in ROCm — vLLM supports, some manual calibration

STEP 6 — TCO calculation baseline:
  $/TFLOPS (BF16): card_price / peak_BF16_TFLOPS
  $/TB HBM BW:     card_price / hbm_bandwidth
  $/GB HBM:        card_price / hbm_capacity

  Compare on the metric that matters for YOUR workload (from Step 2).
```

3A.8.1 Worked Example: H100 vs MI300X for LLaMA-3 70B Serving

```
ANALYTICAL COMPARISON — H100 SXM5 vs MI300X for 70B BF16 Serving:

HARDWARE SPECS:
  H100 SXM5: 989 TFLOPS BF16, 3.35 TB/s, 80 GB, $65K [ESTIMATED]
  MI300X:    1307 TFLOPS BF16, 5.3 TB/s,  192 GB, $45K [ESTIMATED]

STEP 1 — Model fit:
  LLaMA-3 70B BF16: 140 GB
  H100: 140 > 80 → requires TP=2 (2 × H100, $130K, 2 GPUs)
  MI300X: 140 < 192 → single GPU ($45K, 1 GPU)

STEP 2 — Cost per deployment unit:
  H100 option: 2 × H100 = $130K for 140 GB serving capacity
  MI300X:      1 × MI300X = $45K  (3× cheaper for same workload!)

STEP 3 — Theoretical decode throughput (batch=1):
  H100 × 2 (TP=2): 2 × 3.35 TB/s effective - AllReduce = ~5.9 TB/s net
    Decode: 5,900 GB/s / 140 GB = 42.1 tok/s theoretical → ~30 tok/s practical
```

```
  MI300X (1 GPU): 5.3 TB/s
    Decode: 5,300 GB/s / 140 GB = 37.9 tok/s theoretical → ~27 tok/s practical

STEP 4 — Communication overhead:
  H100 × 2: 160 NVLink AllReduces per step
    = 160 × ~0.26 ms = 42 ms communication overhead per step
  MI300X: 0 AllReduces (single GPU)
    = 0 ms communication overhead

STEP 5 — Practical throughput at batch=32:
  At batch=32, H100 has slightly higher effective BW with TP=2:
    ~57 tok/s vs MI300X ~48 tok/s
  At batch=128+, H100 × 2 pulls ahead on sustained throughput.

DECISION FRAMEWORK:
  Small/medium traffic (< 50 requests/sec): MI300X wins on cost (3× cheaper)
  Large traffic with TP=2 viable: H100 × 2 wins on throughput above batch=128
  Latency-sensitive (P99 TTFT < 200ms): MI300X wins (no AllReduce overhead)
  Budget-constrained: MI300X clear winner
```

3A.9 Key Takeaways and Review Questions

```
CHAPTER 3A — WHAT EVERY ARCHITECT MUST REMEMBER:

1. The memory hierarchy is the design constraint, not the compute.
   HBM bandwidth (3.35-8 TB/s) is the binding limit for nearly every
   LLM inference workload. Raw TFLOPS are irrelevant when memory-bound.

2. Tensor Cores require aligned inputs — violations are silent.
   FP8/BF16 Tensor Cores require dimensions divisible by 16 (BF16) or
   128 (FP8). Misaligned GEMM shapes run at 30-40% of peak.
   Always validate shapes with Nsight Compute before declaring a kernel slow.

3. The Transformer Engine is the single most important H100 innovation.
   It dynamically switches per-layer between FP8 and BF16, delivering ~1.9×
   GEMM throughput with <0.2% quality loss and zero code changes.

4. TP must stay within the NVLink domain — no exceptions.
   TP AllReduce runs 160× per step. At InfiniBand latency: 432 ms overhead.
   At NVLink: 42 ms. TP=8 on one node. Never TP=16 across two nodes.
```

```
5. AMD's memory advantage is real and model-size-specific.
   MI300X/MI325X (192-256 GB HBM) eliminates TP for 70B FP16 (needs 2 H100s).
   The benefit is AllReduce elimination, not raw bandwidth.
   For FP8 weights (70 GB), the advantage disappears.

6. SXM vs PCIe is a NVLink question, not a compute question.
   SXM: 900 GB/s NVLink, 700W, mandatory for TP and high-MFU training.
   PCIe: 128 GB/s, 350W, viable for TP-free inference and development.

7. The H200 teaches the most important architectural lesson of this generation.
   Same Tensor Core count as H100, 141 GB vs 80 GB, 4.8 vs 3.35 TB/s.
   Result: ~40% more inference throughput for 70B at zero compute improvement.
   Memory bandwidth, not FLOPs, is the LLM inference bottleneck.

HARDWARE NUMBERS TO MEMORIZE [SHIPPED]:
  H100 SXM5:  989 TFLOPS BF16, 3.35 TB/s HBM3,  80 GB,  700W, ridge ~295
  H200 SXM5:  989 TFLOPS BF16, 4.8 TB/s HBM3e,  141 GB, 700W, ridge ~206
  MI300X:     1307 TFLOPS BF16, 5.3 TB/s HBM3,  192 GB, 750W, ridge ~247
  B200 SXM:   4500 TFLOPS FP8, 8.0 TB/s HBM3e,  192 GB, 1000W [SHIPPED]
  NVLink 4.0: 900 GB/s per GPU (H100)
  NVLink 5.0: 1,800 GB/s per GPU (B200)
```

Review Questions

Difficulty: Foundational

Question 1 — GPU Generation Selection: Your team must serve LLaMA-3 70B in BF16 at a target of 2,000 tokens/sec with P99 TTFT < 400ms on a fixed budget of 8 GPUs. You can choose: 8×H100 SXM5 or 8×MI300X. (a) What is the required tensor parallelism degree for each platform to fit the model? (b) Calculate AllReduce overhead per decode step for the H100 configuration. (c) Which platform wins on this specific workload? What workload characteristic would flip the decision?

Expected answer: (a) H100 (80 GB): 140 GB model needs TP=2 per instance → 4 serving instances across 8 GPUs (each uses 2 GPUs). MI300X (192 GB): model fits single-GPU → 8 serving instances, no TP. (b) H100 TP=2 AllReduce: 160 calls/step × 134 MB × 2(N-1)/N / 900 GB/s = 160 × 0.26 ms = 42 ms/step overhead. (c) For TTFT-sensitive (P99 < 400ms): MI300X wins because 8 independent serving instances with no AllReduce vs 4 instances with 42ms overhead per step. Flip decision: if the workload is batch=256+ sustained training throughput (H100 × 2 aggregate BW 6.7 TB/s vs MI300X single 5.3 TB/s) or if ROCm software maturity is a blocker for custom kernels.

Difficulty: Intermediate

Question 2 — Arithmetic Intensity and Tensor Core Utilization: A colleague profiles a custom attention variant on H100 and reports "Tensor Core utilization is 72% but throughput is only 55% of equivalent FlashAttention." Without additional profiling data, identify the three most likely root causes. For each, state which Nsight Compute counter confirms it and what the remediation is.

Expected answer: (1) Misaligned GEMM shapes: 72% TC utilization means 28% of TC cycles wasted on partial tiles. Confirm: ncu pipe_tensor_cycles_util vs sm_active_cycles ratio. Fix: pad matrix dimensions to nearest multiple of 16 (BF16) or 128 (FP8). (2) Excessive HBM traffic vs SMEM usage: attention is materializing the full S×S score matrix to HBM rather than keeping it in SMEM. Confirm: ncu l1tex_t_bytes_pipe_lsu_mem_global_op_ld (HBM bytes read). Fix: use FlashAttention's tiled SMEM approach — the 55% throughput gap is consistent with 5-10× more HBM traffic per token. (3) Suboptimal warpgroup scheduling (WGMMA pipeline stall): 72% TC util with 55% throughput suggests ~23% of compute cycles are stalled waiting for the next tile to arrive in SMEM. Confirm: ncu smsp_warp_issue_stalled_mio_throttle_per_warp (shared memory throttle). Fix: double-buffer SMEM tiles — prefetch next tile while computing on current one.

Difficulty: Principal Level

Question 3 — NVLink Topology and Parallelism Design: You are designing a 256-GPU H100 SXM5 cluster (32 nodes × 8 GPUs) for LLaMA-3 70B training (80 layers, d_model=8192). Specify TP, PP, and DP degrees. For the TP configuration, calculate per-layer AllReduce message size, NVLink round-trip time, and total TP communication overhead per forward pass. Now explain what happens if a firmware bug degrades NVLink to 450 GB/s on 4 of your 32 nodes — how do you detect it before it costs a week of training?

Expected answer: Parallelism config: TP=8 (within each 8-GPU node, NVLink), PP=4 (across nodes, IB), DP=8 (across PP groups). Total = 8×4×8 = 256. ✓ Per-layer TP AllReduce: activation size = 1 × seq × d_model × 2B = 1 × 4096 × 8192 × 2 = 67 MB per tensor. Two AllReduces per block (attention + FFN): 134 MB total per layer. NVLink ring AllReduce (8 GPUs): time = 2×(8-1)/8 × 134 MB / 900 GB/s = 0.261 ms per layer. Total 80 layers: 80 × 0.261 = 20.9 ms TP overhead per forward pass. Degraded NVLink detection: (1) Pre-flight: run nccl-tests allreduce_perf on all 8-GPU groups before training. At 450 GB/s, AllReduce time doubles to ~0.52 ms/layer → step time increases by ~20 ms detectably. (2) Production: monitor per-node AllReduce latency via NCCL_DEBUG_SUBSYS=NET NCCL logging during first 100 steps. Nodes with degraded NVLink show 2× longer AllReduce vs peer nodes in same PP stage. (3) DCGM: dcgmi dmon -e 1011,1012 (NVLink TX/RX bytes/sec) — degraded nodes show 50% lower NVLink utilization vs healthy nodes. Alert threshold: NVLink BW < 800 GB/s during AllReduce.

Difficulty: Principal Level

Question 4 — Blackwell Architecture Tradeoffs: The B200 uses a dual-die MCM design at 1,000W TDP. A peer argues "B200 is two H100s glued together — not a real advance." Construct the strongest counter-argument covering: what NVLink-C2C enables beyond two separate H100s, why FP4 is an architectural advance rather than a software trick, and what 8.0 TB/s memory bandwidth implies for 70B decoding.

Expected answer: NVLink-C2C: Two separate H100s connected by NVLink communicate at 900 GB/s with ~1 μs latency through the NVSwitch. The NV-HBI between B200 dies operates at 10 TB/s with sub-cycle (effectively zero) cross-die latency — the two dies are coherently addressable as one. This means WGMMA instructions can source operands from SMEM on either die without software coordination. A "two H100s glued together" connected by NVLink cannot do this — the programmer must explicitly manage which data lives on which GPU. FP4 architectural advance: FP4 is not a software quantization scheme applied to BF16 hardware. It requires new 4-bit multiply-accumulate hardware in the Tensor Core matrix unit — the MMA tile must support 4-bit integer arithmetic with FP scaling. This is analogous to how FP8 required E4M3/E5M2 hardware circuits in Hopper, not just bit truncation. The 9 PFLOPS FP4 rate is a hardware circuit count increase, not an instruction reuse. 70B decode at 8.0 TB/s: theoretical max = 8,000 GB/s / 140 GB = 57.1 tok/s per GPU (single card, model fits). At 70% utilization: ~40 tok/s. That is 1.7× H200 (24 tok/s) and 2.5× H100×2 (16 tok/s per GPU pair). For a 1,000-GPU serving cluster, 40 vs 16 tok/s/GPU = 2.5× more capacity at same hardware count. The memory bandwidth alone, not FP4, drives inference-focused TCO.

Further Reading

1. *NVIDIA H100 Tensor Core GPU Architecture Whitepaper* (2022). `resources.nvidia.com/en-us-tensor-core` Read Sections 2 (SM architecture), 3 (Tensor Cores), and 5 (NVLink). Know the numbers on pages 8-12 cold.
2. *NVIDIA Blackwell Architecture Technical Brief* (2024). `www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/` Focus on: NV-HBI, 2nd-gen Transformer Engine, NVLink 5.0, FP4 formats.
3. *AMD CDNA 3 Architecture Whitepaper* (2023). `www.amd.com/en/technologies/cdna` Critical for AMD users: the XCD/MCD die count, Infinity Fabric, MI300X vs MI300A.
4. *In-Datacenter Performance Analysis of a Tensor Processing Unit* — Jouppi et al. (2017). `dl.acm.org/doi/10.1145/3079856.3080246` The original TPU paper. The roofline analysis methodology is timeless.
5. *Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking* — Jia et al. (2018). `arxiv.org/abs/1804.06826` Best detailed analysis of SM internals, warp scheduling, and memory hierarchy.

— End of Chapter 3A —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved